

Supplementary material: Full List of Syntax Conventions of Python

**Yuanheng Li^{1*}, Zhuoyang Chen^{1*}, Xiaoyun Liu¹, Yuhao Wang¹,
Mingwei Liu², Yang Shi¹, Kaifeng Huang^{1†}, Shengjie Zhao¹**

¹Tongji University

²Sun Yat-sen University

*Both authors contributed equally to this work.

†Kaifeng Huang is the corresponding author.

Table 1: List of the Collected Syntax Conventions ([SP], [BR], and [IND] denote the space, line break, and indentation, resp.)

Category	Syntax Node	Conditional Token	Consequent Token
Data Model	Lists, Slice	[	]
	Dict, Set	{	}
	String, Bytes	"", ''	"", ''
	Object	<i>identifier</i>	.
	Tuple	(	)
Expressions	call	<i>identifier</i> (	)
	lambda	lambda <i>params</i>	:
	conditional	<i>expr</i> if <i>cond</i>	else
	comprehension	<i>expr</i> for <i>target</i>	in
		<i>expr</i> for <i>target</i> in <i>iterable</i>	], },)
	chained comparison	<i>expr</i> <i>comp_op</i> <i>expr</i>	<i>comp_op</i>
Single Statements	import	import <i>module</i>	as
		from <i>module</i>	import
		if <i>assignment_expression</i>	:
Compound Statements	if	if <i>assignment_expression</i> :	[SP], [BR], [IND]
		if <i>assignment_expression</i> :	elif, else
	for	for <i>target_list</i>	in
		for <i>target_list</i> in <i>starred_list</i>	:
		for <i>target_list</i> in <i>starred_list</i> :	[SP], [BR], [IND]
	try	try	:
		try:	[SP], [BR], [IND]
		try: <i>suite</i>	except, except *
		try: <i>suite</i> except <i>expression</i>	as
		try: <i>suite</i> except <i>expression</i> as <i>name</i>	:
		try: <i>suite</i> except <i>expression</i> as <i>name</i> :	[SP], [BR], [IND]
		try: <i>suite</i> except <i>expr</i> as <i>identifier</i> : <i>suite</i>	finally, else
	with	with <i>with_item</i>	as
		with <i>with_item</i> as <i>target</i>	:
		with <i>with_item</i> :	[SP], [BR], [IND]
		with <i>with_item</i> as <i>target</i> :	[SP], [BR], [IND]
	class	class <i>classname</i>	:
		class <i>classname</i> :	[SP], [BR], [IND]
		class <i>classname</i> (<i>identifier</i>	:',)
		class <i>classname</i> (<i>identifier</i>)	:
	function	def <i>funcname</i> (	self,)
		def <i>funcname</i> (<i>identifier</i>	:',)
		def <i>funcname</i> (<i>parameters</i>)	'->'
		def <i>funcname</i> (<i>parameters</i>) [-> <i>type</i>]	:
		def <i>funcname</i> (<i>parameters</i>):	[SP], [BR], [IND]
	while	while <i>condition</i>	:
		while <i>condition</i> :	[SP], [BR], [IND]
		while <i>condition</i> : <i>suite</i>	else
	match	match <i>subject</i>	:
		match <i>subject</i> :	[SP], [BR], [IND], case